

Lab2

2026年2月23日 星期一 21:20

1. SmartCard Class

Code:

```
1 public class SmartCard { 7个用法
2     private String ownerName; // stores the full name of the card owner 2个用法
3     private boolean isStaff; // true if the cardholder is staff, false if student 3个用法
4
5     public SmartCard(String ownerName) { 2个用法
6         this.ownerName = ownerName; // 'this.ownerName' refers to the field, 'ownerName' (right) is the parameter
7         this.isStaff = false; // Every new card defaults to student status
8     }
9
10    public String getOwner() { 0个用法
11        return ownerName; // Returns the stored owner name string to the caller
12    }
13
14    public boolean isStaff() { 0个用法
15        return isStaff; // Returns the current boolean value of the isStaff field
16    }
17
18    public void setStaff(boolean isStaff) { 1个用法
19        this.isStaff = isStaff; // 'this.isStaff' is the field, 'isStaff' (right) is the method parameter, 'this' distinguishes them
20    }
21 }
22
```

2. CardLock Class

Code:

```
1 public class CardLock { // Public class to be used by Door, Simulation 5个用法
2
3     private SmartCard lastCardSeen; // Private field to hold a reference to the most recently swiped SmartCard object 4个用法
4     private boolean allowStudents; // students can unlock the door when true 4个用法
5
6     public CardLock() { 2个用法
7         this.allowStudents = false; // Only staff may unlock by default; students are blocked until toggleStudentAccess() is called
8     }
9
10    // Stores the reference to the passed SmartCard so isUnlocked() can check it later
11    public void swipeCard(SmartCard card) { 2个用法
12        this.lastCardSeen = card; // Overwrites any previously swiped card; only the LAST card matters for unlock checks
13    }
14
15    public SmartCard getLastCardSeen() { 0个用法
16        return lastCardSeen; // Returns the reference to the last SmartCard
17    }
18
19    public void toggleStudentAccess() { 0个用法
20        allowStudents = !allowStudents; // inverts the current value
21    }
22
23    public boolean isUnlocked() { 1个用法
24        // First check: impossible to get a NullPointerException if no card was swiped yet
25        // Second check (inside parentheses): uses || (OR) so EITHER condition being true is enough to unlock
26        return lastCardSeen != null && (lastCardSeen.isStaff() || allowStudents);
27        // guard against null / staff card grants access / student access mode also grants access
28    }
29 }
30
```

3. Simulation class

Code:

```
1 public class Simulation {
2     public static void main(String[] args) {
3         Door researchLab = new Door("Research Labs"); // Create a Door object, the Door constructor also creates a default CardLock
4
5         CardLock labLock = new CardLock(); // Constructs a new CardLock with allowStudents = false by default
6
7         researchLab.attachLock(labLock); // Passes the labLock reference to the Door; Door and Simulation now share the SAME CardLock object
8
9         SmartCard studentCard = new SmartCard("Student A"); // Constructor sets isStaff = false
10        SmartCard staffCard = new SmartCard("Staff B"); // Also starts as false; we must explicitly promote it to staff below
11
12        staffCard.setStaff(true); // Sets isStaff field inside staffCard to true; this card can now unlock staff-only doors
13
14        labLock.swipeCard(studentCard); // Registers studentCard as the lastCardSeen on the CardLock
15        System.out.println("Door Open (Student)? " + researchLab.openDoor());
16        // openDoor() to lock.isUnlocked() to studentCard.isStaff() returns false AND allowStudents is false so that it will return false
17
18        labLock.swipeCard(staffCard); // Overwrites lastCardSeen with staffCard
19        System.out.println("Door Open (Staff)? " + researchLab.openDoor());
20        // openDoor() to lock.isUnlocked() to staffCard.isStaff() returns true so that overall returns true
21    }
22 }
23
```

Output:

Output:

```
Door Open (Student)? false
Door Open (Staff)? true
```

Questions for Logbook

- What is the difference between a class and an object?

A class is like a blueprint or template, it defines what attributes and methods something should have, but it doesn't actually exist in memory yet. And object is a real instance created from that blueprint. For example, SmartCard is the class here, but when I write `new SmartCard("Student A")`, that creates an actual object in memory. I can also create many objects from the same class. In this lab, SmartCard is the class, while studentCard and staffCard are two separate objects made from it.

- Why use a constructor instead of direct variable assignment?

Because a constructor makes sure the object is set up correctly when it's created. If I just did direct assignment like:

```
javaSmartCard card = new SmartCard();
card.ownerName = "Student A";
```

I may forget to set one of the fields, making the object incomplete. With a constructor, all the required values are passed in at creation time so I can't accidentally skip them. In this lab, the SmartCard constructor automatically sets `isStaff = false` for every new card, so I don't have to remember to do that manually every time.

- What issues arise if `isStaff` is modified directly instead of using `setStaff()`?

If `isStaff` were public and could be changed directly like `studentCard.isStaff = true`, it might be set to an invalid state. Second, if I later wanted to add validation, I have to find and update every place in the code that does direct assignment, which could be a lot of places. By using `setStaff()`, all the logic is in one place. This is called encapsulation, keeping the fields private and control access through methods so the class can protect its own data.

2. Using ArrayLists

1. User Class

Code:

```
1 public class User { 11个用法
2
3     private String username; // unique login identifier for this user 2个用法
4     private String userType; // "user", "editor", "admin" 3个用法
5     private String name; // human-readable display name of this user 2个用法
6
7     public User(String username, String userType, String name) { 4个用法
8         this.username = username; // Assigns the constructor parameter 'username' to the instance field 'this.username'
9         this.userType = userType; // Assigns 'userType' parameter to the instance field; 'this.' distinguishes field from parameter
10        this.name = name; // Assigns 'name' parameter to the instance field 'this.name'
11    }
12
13    public String getUsername() { 3个用法
14
15        return username; // Returns the stored username
16    }
17
18    public String getUserType() { 3个用法
19
20        return userType; // Returns the stored userType
21    }
```

```

22
23 >     public String getName() { return name; // Returns the stored name string }
26
27     public void setUserType(String userType) { 0个用法
28         this.userType = userType; // Overwrites the current role; 'this.userType' = field, 'userType' = parameter
29     }
30 }

```

2. UserGroup Class

Code:

```

1 import java.util.ArrayList; // using ArrayList<>
2
3 public class UserGroup { 2个用法
4
5     private ArrayList<User> users; // Private dynamic list (ArrayList) that stores User objects 9个用法
6
7     public UserGroup() { 1个用法
8         users = new ArrayList<>(); // creating an empty list
9     }
10
11     public void addSampleData() { 1个用法
12         users.add(new User( username: "fj3", userType: "user", name: "Francis")); // Creates a User object inline and adds it to the list, index 0
13         users.add(new User( username: "amn", userType: "admin", name: "Amn")); // Admin user, index 1
14         users.add(new User( username: "cm2", userType: "admin", name: "Cameron")); // Admin user, index 2
15         users.add(new User( username: "pm5", userType: "editor", name: "Peter")); // Editor user, index 3
16     }
17
18     public User getUser(int index) { return users.get(index); // retrieving the element at the given index 0 }
19
20     public void printUsernames() { 1个用法
21         for (User u : users) { // 'u' is a temporary variable referencing each User in turn
22             System.out.println(u.getUsername() + " " + u.getUserType()); // Concatenates username and userType with a space
23         }
24     }
25
26     public void removeUser(String username) { 0个用法
27         users.removeIf( User user -> user.getUsername().equals(username));
28         // 'removeIf' goes through each element and removes it if the lambda (condition) returns true
29         // 'user ->' declares a lambda parameter named 'user' (each element in the list)
30         // '.equals()' performs a content comparison; '==' would compare object references, not string values
31     }
32
33     public ArrayList<User> getUsers() { return users; // Returns the reference to the internal ArrayList }
34 }

```

3. Main Java

Code:

```

1 import java.util.ArrayList; // Import ArrayList to declare the 'administrators' list
2
3 public class Main {
4     public static void main(String[] args) { // initialises an empty ArrayList UserGroup
5         UserGroup group = new UserGroup(); // 'group' is a reference variable pointing to the new UserGroup object on the heap
6
7         group.addSampleData(); // Calls addSampleData() on the 'group' object to populate and adds 4 User objects to internal list
8
9         System.out.println("All Users:");
10
11         group.printUsernames(); // Call printUsernames() to loop through every user and print "Username userType"
12
13         ArrayList<User> administrators = new ArrayList<>(); // Declares and initialises an empty list; <User> is the generic type parameter (type-safe)
14
15         // Loop through every User in the group's list and check if they are an admin
16         for (User user : group.getUsers()) {
17             if (user.getUserType().equals("admin")) { //compares the string content of userType
18                 administrators.add(user); // Adds the current admin User reference to admin list
19             }
20         }
21
22         System.out.println("\nAdministrators:");
23
24         // Loop through the filtered administrators list and print each admin's username and type
25         for (User admin : administrators) {
26             System.out.println(admin.getUsername() + " " + admin.getUserType()); // Prints "amn admin", then "cm2 admin"
27         }
28     }
29 }

```

Output:

```

All Users:
fj3 user
amn admin
cm2 admin
pm5 editor

```

```
Administrators:|
amn admin
cm2 admin
```

3. HackerRank

1. Java ArrayList

Code:

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
11         Scanner sc = new Scanner(System.in);
12         int n = sc.nextInt();
13         int[][] lines = new int[n][];
14         for (int i = 0; i < n; i++) {
15             int count = sc.nextInt();
16             lines[i] = new int[count];
17             for (int j = 0; j < count; j++) {
18                 lines[i][j] = sc.nextInt();
19             }
20         }
21         int q = sc.nextInt();
22         for (int i = q; i > 0; i--) {
23             int x = sc.nextInt() - 1;
24             int y = sc.nextInt() - 1;
25             if (x < 0) {
26                 System.out.println("ERROR!");
27             } else if (x >= n) {
28                 System.out.println("ERROR!");
29             } else if (y < 0) {
30                 System.out.println("ERROR!");
31             } else if (y >= lines[x].length) {
32                 System.out.println("ERROR!");
33             } else {
34                 System.out.println(lines[x][y]);
35             }
36         }
37     }
38 }
```

Output:

Submitted a few seconds ago • Score: 1.00

Status: Accepted

✓ Test Case #0
✓ Test Case #3

✓ Test Case #1
✓ Test Case #4

✓ Test Case #2
✓ Test Case #5

Testcase 0 ✓

Testcase 1 ✓

Congratulations, you passed the sample test case.

Click the **Submit Code** button to run your code against all the test cases.

Input (stdin)

```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
```

```
5
1 3
3 4
3 1
4 3
5 5
```

Your Output (stdout)

```
74
52
37
ERROR!
ERROR!
```

Expected Output

```
74
52
37
ERROR!
ERROR!
```

2. Java Iterator

Code:

```
7
8     Object element = it.next(); // get the next element
9     if(element.equals("###")) // stop exactly when we hit "###"
```

Output:

Testcase 0 

Congratulations, you passed the sample test case.

Click the **Submit Code** button to run your code against all the test cases.

Input (stdin)

```
2 2
42
10
hello
java
```

Your Output (stdout)

```
hello
java
```

Expected Output

```
hello
```

java

Submitted a minute ago • Score: 1.00

Status: Accepted



Test Case #0



Test Case #1



Test Case #2